

PLAYBOOK (ADVANCED)

Client AI Agents: the Agentic Foundation

The nine layer standard every Easyworks client agent is built and inspected against

EW-TECH-10 · v1.0 · 2026-09-10

Owner: Brad Palmer · For: Automation Devs, with Brad



WHAT THIS IS

This is the design standard for any client AI agent: an agent that reads a client's system of record, decides what matters, remembers the client, and acts with the client in control of every write. You use it to scope an agent at the audit, to build it layer by layer, and to inspect and score it. It produces an agent that is built against the layers, tracked against the layers, and sold against the layers.

DONE MEANS

Every layer has a maturity score with evidence, no write happens without an approval gate and a read back, the agent runs 24/7 off a box Brad deployed, and the leverage is stated in the client's own numbers.

01 The whole flow



The two gold boxes are Brad's: scope approval and deployment to the client box.

Brad	Automation Dev	Technical Builder	Client
Architects the agent. Approves scope. Deploys to and operates the client boxes. Holds every secret. Signs off each layer.	Builds the layers in the agent repo against this standard. Writes the tests. Never touches a client box.	Supplies the GHL side: data exports, fields, tags, workflows the agent reads or triggers. See EW-TECH-04 and 05.	Names the system of record, approves each write, trains the memory in plain language, judges the leverage.

A chatbot on a CRM is a demo. A durable agent is a stack of layers, each with one job, that turns the client's raw system of record into something that reads their book, decides what matters, remembers them, and acts safely. Build bottom up. A layer is only as trustworthy as the one below it.

The reference implementations are two: a real estate client's brain on their CRM, and the Easyworks Brain on GHL. Both follow this standard. Every new client build either fills a gap in this document or adds a pattern to it. That is how the foundation compounds.

RULE

This playbook is for an Automation Dev working with Brad. It is not a solo build. If you are reading this without a task from Brad that names the client and the layer, stop and ask.

02 The stack: nine layers

Layer	Job	Done when	Inspect question
01 Data	Snapshot the client's whole system of record locally and keep it current inside their API budget.	Full pull lands every record in a timestamped local store. Incremental sync keeps it fresh with far fewer calls. Every number the agent quotes traces to a snapshot.	Is there a snapshot? How stale can it get? Does sync fit the rate limit?
02 Signal and Ranking	Turn data into what matters today, scored from the client's own outcomes.	A daily list of the few worth acting on, each with a reason. Weights derived from their real results. Deadlines detected.	Reason per item? Score from their outcomes or a guess? Deadlines caught?

Layer	Job	Done when	Inspect question
03 Memory	Remember preferences, voice, rules and corrections. Client trainable.	Rules persist across sessions. The client adds a rule in plain language and it sticks. Corrections resolve to something enforceable.	Can the client train it alone? Do corrections persist and enforce?
04 Reasoning	The agent loop: a bounded tool set over the snapshot, answering like a person.	Few sharp tools. Turn cap. Conversation memory. Graceful failure that answers from what it has, never a raw error. Plain text out.	Scoped or sprawling tools? What happens at the turn limit?
05 Action	Change the system of record safely, reversibly, provably.	Every write is proposed, approved by the client in one tap, applied, then verified by reading the record back. Prior state stored. A silent no op counts as a failure.	Anything write without approval? Verified by read back or trusted on a 200?
06 Transport	Reach the client where they already are and read their replies.	One live channel (SMS, Telegram, email) that both sends and receives. Replies feed the reasoning layer. Bursts coalesce, opt out honoured.	New app needed? Can it receive? Are replies read?
07 Operations	Run 24/7 without a laptop, self heal, deploy in one command.	Cloud box under AMPCOS, service timers, watchdog, nightly backup, off box heartbeat, one command deploy, pull only.	Survives a reboot? Watchdog and backup? Deploy one command?
08 Safety	Never harm the data, the contacts or the trust.	Approval gate on every public action. No double contact. DNC respected. Quiet hours. Secrets least privilege and out of the repo. Security pass on other feeds.	Could two systems text one contact? Are secrets scoped?
09 Leverage	Turn the stack into compounding business leverage in the client's numbers.	Output stated as hours reclaimed, dormant value surfaced, work cleared, deals recovered. Growing review over review.	Stated in their numbers? Compounding or just more features?

BOTTOM UP

Build 01 to 08 in order. 09 is not built, it is measured. Ranking is noise without clean data. Action is dangerous without safety.

03 Scoping: walk the layers at the audit

Before an agent is quoted, walk the eight layers with the client's stack in mind. The answers go on the scope sheet. Anything a client wants beyond a layer's Level 2 is a separate priced phase and is named as such, never bundled.

Scoping checklist, layers 01 to 05

- ☐ 01 What is their system of record? Does it have an API, what is the rate limit, is the limit per account or per application? Any data residency need?
- ☐ 02 What outcome data exists to rank from (closings, bookings, conversions)? What deadlines matter to them?
- ☐ 03 What preferences, voice and rules must it hold from day one?
- ☐ 04 What are the three to five things it must do on command? Which two questions will they ask most?
- ☐ 05 What writes are in scope? Are they all reversible? Will the client approve each one?

Layers 06 to 09

- ☐ 06 What channel do they already live in? Can we send and receive there?
- ☐ 07 Standard box, or constraints?
- ☐ 08 Does their platform have its own AI (double contact risk)? Any other tool with a live feed of their data?
Compliance: consent rules for texting, residency?
- ☐ 09 Which of their numbers will show the leverage? Baseline them now.

BUILD FROM THE DATA

Probe the snapshot before designing a feature. The stated pain is often not where the data says the pain is. On the reference build the top ask was deadline tracking, and the CRM held no deadline data at all. The absence was the pain, and the right build was a tracker fed in plain language, not a scanner.

04 Build order and the safety rules

1**Data first**

Full pull to a local snapshot. Then incremental sync with a per run cap. Stamp freshness per collection. A daily health check reports age and request budget.

End state: any figure the agent could quote can be traced to a snapshot row.

2**Signal from their outcomes**

Derive weights from what actually converted for this client. Suppress the owner and the team from every list. Separate real tasks from automated drips.

End state: today's list has a reason per item and matches what a human would pick.

3**Memory the client can train**

Plain language rules that resolve to enforceable facts (a name becomes a phone). Append only, retired entries struck, never deleted.

End state: the client sets a rule by message and the next answer obeys it.

4**Reasoning with a small sharp tool set**

The two most asked questions get deterministic tools, never free form queries. Turn cap. Conversation history persisted per channel. Plain text out.

End state: an eval suite built from the client's real messages passes.

5**Action behind an approval gate**

Propose one change, approval card, apply, read back. Store prior state. Count a silent no op as a failure.

End state: a test write with a wrong field name is reported as failed, not succeeded.

6**Transport on their channel**

Bridge the channel they use, do not rebuild it. Persist a reply before sending. Poll delivery status. Drain quiet hour queues per channel.

End state: a message in, an answer out, a reply read back in, on the real channel.

7**Operations, with Brad**

You supply service and timer definitions and a one command deploy script in the repo. Brad provisions the box and runs the deploy.

End state: a smoke turn passes after deploy and hourly from the watchdog.

8**Safety as a standing job**

Weekly diff of team seats, the platform's own AI settings, open ports and expiring credentials against a baseline.

End state: the check runs on a timer and alerts on drift.

The safety layer rules

APPROVAL GATES BEFORE ANY CLIENT WRITE

No write to the client's system and no message to a contact without an approval card the client taps. The executor checks the approved batch is the same bytes it runs. Suppression rules are never overridable.

VERIFY BY READ BACK

A 2xx is not proof. Re read the record after every write and confirm the change took. If it did not, it failed and the client is told.

NO DOUBLE CONTACT

The client's own platform assistant and our agent never message the same contact the same day. Suppress the platform's assistant on any contact our agent is working.

DNC

Do not contact flags are respected and settable by the client in plain language. An opted out sender is never queued. Compliance keywords (stop, help) are handled in code, not by the model.

QUIET HOURS

The client can reach the agent 24/7 and always gets a live reply. The agent never initiates between 21:00 and 08:00 client local, and never answers a backlog after a deploy or downtime. Reactive replies fire only on a fresh inbound.

SECRET HYGIENE

Each box gets only the keys its own code reads, never the whole secrets file. Secrets never enter a repo, a task, a Loom or an AI tool. Least privilege, scoped per box.

05 **The maturity model: score every layer**

Rate each of the nine layers 0 to 3. Track it per client in the Agent Maturity Tracker. The next move column on the tracker is the build backlog. The total out of 27 is the number Brad watches.

Level	Meaning	Evidence you attach
0 None	Not built.	Nothing.
1 Built	Exists, runs manually or on request.	A run log or a screenshot from a manual run.
2 Live	Running in production, assisting the client daily.	Timer or service status, a real turn from the client, the client's own confirmation.
3 Autonomous	Runs standing workflows inside client set guardrails and reports what it did.	The guardrail the client set, the standing job, and its report to the client.

The engagement arc maps onto the model. Build gets layers to 1 or 2. Train and tune pushes them toward 2. Hand over is the climb to 3 on the layers the client trusts. The climb is the leverage curve, and that is what the client renews on.

RULE

Score with evidence, not optimism. A layer at 2 with no client turn in the last week is a 1. Brad reconciles the tracker at every review.

06 **Runtime, constraints and gotchas**

Why custom runtime instead of n8n or Make

A visual workflow tool is excellent for fixed wiring: when X, do Y, then Z across many SaaS connectors. An agent is not a fixed workflow. It is a model deciding in a loop which tools to call and adapting to what it finds. A workflow tool can call a model as one node; it cannot cheaply be the loop, and you end up writing custom code nodes to reach what the runtime already does. So the runtime stays the agent core: owned, no per execution fee, full control. If a visual layer is ever wanted for non agentic wiring, run it alongside and have it call our endpoints, never as a replacement.

Constraints that bind regardless of tooling

- Some CRMs rate limit per application, not per account. Every client shares one budget. Stagger sync times and cap per run volume.
- SMS to US numbers needs carrier registration filed under a real entity. It takes days, it is not code, and it is not skippable. New client numbers go through GHL with registration in the client's name.
- Compliance keywords are handled in the transport code, never by the model. Opted out senders are never queued.
- The box only ever pulls from our public endpoints. No inbound port. The attack surface is SSH alone.
- Access to the client's platform is granted by the client and revocable by them. That is the correct trust model and we sell it that way.
- Canadian data residency under AMPCOS for the boxes.

Recurring gotchas

Gotcha	What to do instead
Spreading an unbounded array as function arguments overflows the stack at scale. It once killed six days of briefs.	Fold or single pass. Never push(...arr) or Math.max(...arr) on data you did not bound.
A fresh box has no snapshot.	Sync creates its own schema. Never assume the table exists.
Netlify caches function bundles.	Deploy with the skip functions cache flag or changes silently do not ship.
Eventually consistent storage decides whether an action runs.	Strong consistency for the approval gate, the message queue, the idempotency store.
Killing by pattern catches your own SSH command.	Kill by exact process name or PID.
Starting a one shot service blocks until it finishes.	Background it or poll the journal. Never let it hang a deploy.
A missing key or import breaks every turn with no alarm.	Status check on deploy: key present and non empty, timers enabled, transport live. One smoke turn after every restart and hourly.
Evals that write pollute the client's memory and create real approval cards.	Ephemeral sessions simulate every write tool.
A smaller model for speed on the client facing brain.	The brain the client talks to runs the model Brad set. Speed comes from deterministic tools, not a weaker model.

07 Deploy rule and definition of done

BRAD DEPLOYS

Contractors never deploy to a client box. You do not hold the SSH key, the box secrets, or the deploy alias, and you should never ask for them. You push to the repo branch named in the task, attach your test results, and Brad deploys. If you find yourself needing box access to finish a task, the task is scoped wrong. Hit Blocked.

Definition of done for a layer

- ☐ The layer's done when list in section 02 is met, with evidence attached to the task.
- ☐ Every write path goes through propose, approve, apply, read back, and a wrong write is reported as failed.
- ☐ The eval suite (client's real messages plus adversarial cases) passes after your change and is in the repo.
- ☐ No secret in the repo, the task, a Loom or an AI tool. Names only in `.env.example`.
- ☐ Service, timer and deploy definitions updated in the repo if the layer needs them. You did not touch the box.
- ☐ The maturity tracker row for the layer has a proposed score with evidence and a next move.
- ☐ Anything this build taught us is written as a pattern in the task for Brad to add to the foundation.
- ☐ Task moved to review. Brad deployed, ran a smoke turn, and marked it done.

When it goes wrong

Symptom	First three checks	Who to tell
The brief or the daily list did not arrive	1. Timer state and last run in the journal. 2. Did sync finish, and is the snapshot fresh? 3. Off box heartbeat and watchdog alerts.	Brad now. Only he can see the box.
The agent quoted a number that is wrong	1. Trace it to the snapshot row. 2. Which tool produced it, from the turn audit log? 3. Freshness stamp on that collection.	Task notes with the trace. Brad the same day if the client saw it.
A write returned success but nothing changed	1. Read back the record. 2. Field name or contract exactly right for this platform? 3. Is the change applied asynchronously?	Task notes. Add the platform quirk to the patterns list.
A contact was messaged twice	1. Is the platform's own assistant active on that contact? 2. Duplicate records in the snapshot? 3. Idempotency on the send path?	Brad now. This is the failure that loses client trust.
The agent texted at night	1. First message inbound or outbound? 2. Was it a backlog drained after a deploy? 3. Quiet hours module in the transport path?	Brad now. Quiet hours is a hard rule.
Rate limit errors from the client's platform	1. Requests this run from the request log. 2. Are two clients syncing at the same minute? 3. Per run cap in place?	Task notes. Brad schedules the stagger.
Eval suite fails after a prompt change	1. Which assertion, and is it tool choice, tone or latency? 2. Did the change touch the intent table? 3. Revert and rerun to confirm cause.	Task notes. Do not ship the prompt until it passes.